otto: Architecture

A browser assistant that understands any web page, fills any form from a saved profile, and remembers forms it has filled. Version 2, October 2026.

1. Goal and principles

otto runs inside the user's own browser, on the tabs where the user is already logged in. It must work on any website and any framework, stay light, keep personal data on the device, and never take an irreversible action on its own. The design follows seven rules:

#	Rule	Reason
1	Nothing runs on a page until the user opens otto.	No slowdown on pages where otto is not used.
2	Pages are read from their HTML structure, layout and styles, not from accessibility labels.	Many sites have no ARIA or labels.
3	Rules decide first; AI is used only for fields the rules cannot decide, in one request.	Fast, cheap, predictable.
4	The AI returns a plan as data; otto's own code carries it out.	Browser stores forbid running AI-written code; easier to check.
5	The AI sees field labels and profile field names, not the user's personal values.	Hidden page instructions cannot leak what the AI never saw.
6	Every value is read back after filling; a field counts only if the value stayed.	Frameworks often ignore or reset values.
7	otto never submits, pays, sends or deletes by itself.	The user stays in control of every final step.

2. Main parts

The system is one shared engine plus a thin layer for each browser. Only the thin layer changes between browsers.

Part	What it does	Where it runs
Engine (core)	Reads the page, finds fields and labels, fills them, checks the result, records element fingerprints.	Inside the web page, injected on demand into each frame.
Planner	Matches fields to the profile (recipe, then rules, then AI), shows a preview, builds the run report.	Extension side panel or popup.
AI providers	Pluggable: on-device model (Gemini Nano in Chrome, Phi-4-mini in Edge, Apple models on Mac/iPhone), small local model, or a cloud model with the user's key.	Chosen at start-up by what the browser offers.
Browser layer	Chromium: side panel and background page. Firefox: sidebar. Safari: popup and in-page panel. Bookmarklet build for browsers without extensions.	One build per browser from one codebase.
Local store	Encrypted profile, recipes, run history, settings.	Browser storage (IndexedDB), on the device.
Optional companion	Real file paths, local models, real keystrokes for stubborn sites.	Small desktop app, opt-in.

3. How a page is read

Every HTML page has elements, positions, styles and a top-most element at each point; otto relies on these. Each layer is tried only if the cheaper one before it was not enough.

Layer	Method	Typical cost
Structure	Walk the page including open and closed shadow DOM; each iframe is read by its own engine copy.	Under 50 ms for a normal page
Clickable and fillable	Native controls, keyboard focus, click handlers (React, Vue, Svelte, jQuery), hand cursor, icon size.	Included above
Visibility	Size, visibility checks and hit testing; hidden trap fields are marked and never filled.	Included above

Layer	Method	Typical cost
Labels	Linked label, then Chrome autofill's own rules (nearby text, tables, lists, legends), then position, then field attributes.	Few ms
Special pages	Flutter accessibility layer, Google Sheets name box, PDFs read as text.	On demand
Visual fallback	Screenshot, small element detector and text recognition, only for canvas apps and unreadable frames.	0.1 to 1 s, rare

4. How a form is filled

Matching. For each field otto tries, in order: a saved recipe; the field's autocomplete hint; a dictionary of names and labels in several languages; and finally one AI request for all remaining fields together. Values are converted to what the field expects (date format, exact option text, country name or code, phone length). Passwords, card numbers, one-time codes, CAPTCHAs and consent or marketing boxes are never filled. If the profile has no value, otto asks instead of guessing.

Filling. Fields are filled one at a time. Each field has a ladder of methods; otto stops at the first one whose result reads back correctly.

Step	Method	Used for
1	Widget handler: set the value the way the browser does, then send the normal input and change events.	Text, selects, dates, checkboxes, radios, sliders
2	Open, type to filter, click the matching option.	Custom dropdowns (react-select, MUI, Ant Design, Radix, Headless UI, Select2)
3	Type character by character.	Masked inputs (phone, dates), rich-text editors
4	Call the widget library's own function inside the page.	Date pickers and editors that ignore events
5	Real keystrokes through the browser debugging interface (Pro build or companion only).	Rare sites that reject simulated input
6	Highlight the field and hand it to the user.	Anything still failing

After the whole form is filled, otto waits briefly, checks every value again and refills any that the page reset. It then scans again, because new questions often appear after an answer.

5. Memory and recipes

A successful fill is saved as a recipe: one step per field with the profile field name and a fingerprint of the element (label, name, type, section, nearby text, position). On the next visit the recipe fills the form with no AI. When the site changes, otto repairs only the steps that broke: first by the field's meaning, then by fingerprint similarity, and only the remaining fields go to the AI. A repaired recipe is kept as a new version and becomes the default only after it fills correctly; older versions can be restored. Steps that could submit or pay are never repaired automatically.

Memory	Holds
Profile	Personal details, address, work, education, links, documents, custom fields. Each field has a sensitivity level and a never-fill switch.
Site memory	Field-to-profile mappings learned per form, including user corrections.
Recipes	Versioned steps with fingerprints and repair history.
Run history	Per field: method used, source, value read back, time, error. Used for debugging and testing.

6. Privacy and safety

Risk	Protection
Personal data reaching the AI	Only field descriptions and profile field names are sent. Values are inserted on the device. Card data never leaves the device.
Instructions hidden in pages	Hidden and near-invisible text is removed; page text is marked as untrusted; actions must use elements that exist; a cap on actions per request.
Unwanted actions	Code-level checks before any submit, payment, send, delete, password or cross-site form; navigation to new sites is blocked during a fill.
Bot traps	Only fields a person can see are filled; CAPTCHAs and one-time codes are handed to the user.

Risk	Protection
Stored data	Profile encrypted with a key from the user's password or passkey, kept in memory only while unlocked, locked after idle time.

7. Performance targets

Measure	Target	Measure	Target
Cost on unused tabs	None	Engine size	Under 60 KB
Scan, normal page	Under 50 ms	Scan, 50,000 elements	Under 300 ms, no freeze over 50 ms
First field from a recipe	Under 100 ms	First field with cloud AI	Under 1.5 s
AI requests, first visit	At most 1 per form step	AI requests, repeat visit	0
Wrong fills	Close to 0	Trap fields filled	0

8. Browser support

Browser	Panel	On-device AI	Notes
Chrome, Edge	Side panel	Gemini Nano, Phi-4-mini	Full feature set; first release
Brave, Vivaldi, Arc, Opera	Side panel or sidebar	WebGPU models	Install from the Chrome Web Store
Firefox (desktop, Android)	Sidebar or overlay	WebGPU or small models	No real-keystroke mode
Safari (Mac, iPhone, iPad)	Popup and overlay	Apple on-device models	Needs Apple developer account and Xcode build
Others (DuckDuckGo, Chrome on Android)	Overlay	None	Bookmarklet or userscript version

9. Known limits

No tool fills every field on every site; widget libraries are a long list that needs ongoing care, and handing a field to the user is part of the design. Sites that accept only real keystrokes need the Pro build or the companion. Hidden page instructions are an unsolved problem industry-wide; otto limits what such an attack can reach rather than claiming to stop it. On-device AI needs a capable computer, so many users will rely on rules plus a cloud model. Browsers are adding similar built-in features, so otto must stand out through reliability on hard forms, recipes that survive site changes, cross-browser support and data that stays with the user.